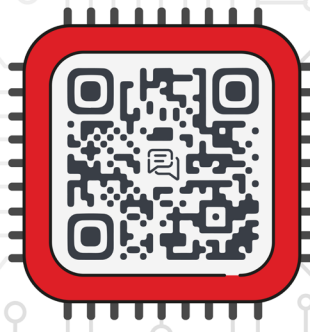




نظري

7



5



750 sp

كلية الهندسة المعلوماتية

السنة الأولى

Local & Anonymous Inner Classes

د. فراس ضعيف

د. معتر النجرس



البرمجة 2

RB Informatics; 07/06/2026

استكمالاً لما تم تناوله في المحاضرات السابقة حول الأصناف الداخلية Inner Classes، تتناول هذه المحاضرة النوعين المتبقين منها، وهما: الأصناف الداخلية المحلية Local Inner Classes، والأصناف الداخلية المجهولة Anonymous Inner Classes.

الكلاسات الداخلية المحلية Local Inner Classes

المفهوم والنطاق Scope

الكلاس الداخلي المحلي هو صنف يتم تعريفه داخل نطاق محدد داخل البرنامج، وغالباً داخل دالة أو تابع Method تابع لصنف آخر. وقد يكون هذا النطاق، من الناحية العامة، كتلة برمجية Block داخل دالة أو باني Constructor أو غيرهما، إلا أن المثال الأكثر شيوعاً يكون داخل الدوال.

- الرؤية والصلاحيات: ينحصر وجود هذا الصنف ورؤيته واستخدامه داخل النطاق الذي عُرِف فيه فقط، فلا يمكن لأي جزء خارج هذا النطاق رؤيته أو إنشاء مثيل Instance منه.
- الهدف البرمجي: يُستخدم هذا الأسلوب عندما تكون هناك حاجة إلى صنف يؤدي وظيفة مؤقتة وخاصة جداً بخوارزمية الدالة الحالية، من غير حاجة إلى إظهاره لبقية أجزاء البرنامج.

مثال توضيحي من الواقع:

يشبه هذا المفهوم وجود أداة مساعدة مؤقتة يستخدمها المهندس داخل مشروع محدد، كبرنامج صغير يقوم بحسابات خاصة أثناء تنفيذ جزء معين من التصميم. تُستخدم هذه الأداة ضمن هذا الجزء فقط، ولا تكون ذات فائدة أو حاجة للاستخدام خارج هذا السياق المحدد.

أمثلة برمجية تطبيقية

1. المثال الأول: منسق النصوص Printer & Formatter

يوضح المثال الآتي كلاساً محلياً وظيفته تطبيق تنسيق محدد على السلسلة النصية الممررة إلى الدالة:

```
class Printer {
    void printMessage(String text) {
        class Formatter {
            String apply(String msg) {
                return "[START] " + msg + " [END]";
            }
        }
    }
}
```

```

    }
    Formatter f = new Formatter();
    System.out.println(f.apply(text));
}
}

```

في هذا المثال، تم تعريف الكلاس `Formatter` داخل الدالة `printMessage`، ولذلك لا يمكن استخدامه أو إنشاء كائن منه خارج هذه الدالة. يقتصر دوره على تنسيق النص الممرر ثم طباعته، وهذا يوضح فكرة حصر الصنف داخل المكان الذي يحتاج إليه فقط.

2. المثال الثاني: حساب التقدير الجامعي `Teacher & GradeCalculator`

يمثل المثال الآتي حالة برمجية لحساب التقدير الجامعي اعتماداً على العلامة المدخلة، وذلك من خلال كلاس محلي يُعرّف داخل الدالة المسؤولة عن عملية التقييم.

```

class Teacher {
    void gradeExam(int score) {
        class GradeCalculator {
            char calculate() {
                if (score > 90) return 'A';
                if (score > 75) return 'B';
                if (score > 60) return 'C';
                if (score > 50) return 'D';
                return 'F';
            }
        }
        System.out.println(new GradeCalculator().calculate());
    }
}

```

يعتمد الكلاس المحلي `GradeCalculator` على المتغير `score` الممرر إلى الدالة `gradeExam`. وبما أن قيمة `score` لا تتغير داخل الدالة، فيُعدّ المتغير `effectively final`، ولذلك يمكن للكلاس المحلي استخدامه.

الكلاسات الداخلية المجهولة `Anonymous Inner Classes`

- المفهوم والآلية: الأصناف الداخلية المجهولة `Anonymous Inner Classes` هي أصناف لا تحمل اسماً صريحاً في الشيفرة البرمجية.
- طبيعة الاستخدام: تُستخدم غالباً عندما تكون هناك حاجة إلى إنشاء كلاس لغرض محدد وسريع، مع استخدامه مباشرة في موضع إنشائه. ويُعرّف الصنف ويُنشأ منه كائن في التعبير البرمجي نفسه.
- الارتباط التوريثي: هذا النوع يكون دائماً مبنياً على كلاس آخر أو على واجهة `Interface`. فإذا كُتب بعد الكلمة `new` اسم كلاس، فإن الكلاس المجهول يكون امتداداً له. وإذا كُتب اسم واجهة، فإن الكلاس المجهول يكون تطبيقاً لها. ولا تُكتب الكلمات المفتاحية `extends` أو `implements` صراحة في هذه الحالة.

```
ParentClassOrInterface obj = new ParentClassOrInterface() {
    // Override methods or define behavior
};
```

في حالة الكلاس العادي، يمكن استدعاء باني الكلاس الأب من خلال الأقواس بعد اسم الكلاس. أما في حالة الواجهة، فلا يوجد باني فعلي للواجهة، وإنما تستخدم الأقواس ضمن الصيغة العامة لإنشاء كائن من كلاس مجهول يطبق تلك الواجهة.

مثال برمجي: وراثة كلاس اعتيادي

يوضح المثال الآتي إعادة تعريف سلوك دالة تابعة لكلاس أساسي دون الحاجة إلى بناء كلاس مستقل يحمل اسماً منفصلاً.

```
class BaseTeacher {
    void evaluateExam() {
    }
}

class Test {
    public static void main(String[] args) {
        BaseTeacher a = new BaseTeacher() {
            @Override
            void evaluateExam() {
                System.out.println("Anonymous behavior");
            }
        };
        a.evaluateExam();
    }
}
```

في هذا المثال، لم يتم إنشاء كلاس جديد باسم مستقل يرث من BaseTeacher، بل تم إنشاء كلاس مجهول في موضع الإسناد نفسه، مع إعادة تعريف الدالة evaluateExam. وعند استدعاء الدالة من خلال المرجع a، يُنفذ السلوك الجديد المكتوب داخل الكلاس المجهول.

تطبيق الواجهات عبر الأصناف المجهولة

تعد هذه الفكرة من النقاط المهمة في فهم علاقة الأصناف المجهولة بالواجهات Interface.

- السؤال العلمي هنا هو: كيف يمكن إنشاء كائن مرتبط بواجهة Interface، مع أن الواجهة لا يمكن إنشاء كائن مباشر منها لأنها تحتوي على دوال مجردة ولا تملك باني كالأصناف العادية؟
- الإجابة: لا يتم إنشاء كائن مباشر من الواجهة نفسها، وإنما يتم إنشاء كائن من كلاس مجهول يطبق هذه الواجهة ويحقق دوالها المجردة في لحظة الإنشاء نفسها. وبذلك يصبح المرجع من نوع الواجهة، بينما يكون الكائن الفعلي من نوع كلاس مجهول يحقق هذه الواجهة.

يوضح هذا المثال فكرة ربط سلوك معين بحدث (Event) وهو "النقر على زر"، وذلك باستخدام واجهة وكلاس مجهول لتحديد هذا السلوك بشكل مرن.

الفكرة الأساسية تقوم على فصل "الزر" عن "ما الذي يحدث عند الضغط عليه"، بحيث يمكن تغيير السلوك بسهولة دون تعديل كلاس الزر نفسه.

```
interface ClickCover {
    void onClick();
}

class Button {
    String name;
    ClickCover actualClickCover;

    Button(String name, ClickCover actualClickCover) {
        this.name = name;
        this.actualClickCover = actualClickCover;
    }

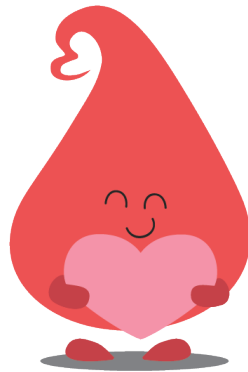
    void click() {
        System.out.println(name + " clicked");
        actualClickCover.onClick();
    }
}

class TestButton {
    public static void main(String[] args) {
        Button a = new Button("abe", new ClickCover() {
            @Override
            public void onClick() {
                System.out.println("Click action");
            }
        });
        a.click();
    }
}
```

في هذا المثال، الواجهة ClickCover تحتوي على الدالة onClick. وبما أن الدوال داخل الواجهات تكون public abstract افتراضياً، فقد تم تعريف الدالة داخل الكلاس المجهول باستخدام public. عند إنشاء الكائن Button، تم تمرير كائن من كلاس مجهول يطبق الواجهة ClickCover. هذا الكلاس المجهول حدّد السلوك الذي سيحدث عند تنفيذ الدالة onClick. وعند استدعاء الدالة click من الكائن a، تتم طباعة اسم الزر أولاً، ثم يُستدعى السلوك الفعلي للنقر من خلال actualClickCover.onClick.

مقارنة بين جميع أنواع الأصناف الداخلية

وجه المقارنة	Member inner class	Static Nested Class	Local Inner Class	Anonymous Inner Class
مكان التعريف	داخل متن الكلاس مباشرة، خارج الدوال	داخل متن الكلاس مباشرة، خارج الدوال	داخل دالة أو نطاق برمجي محدد	داخل دالة أو ضمن تعبير برمجي
وجود اسم للكلاس	نعم، يملك اسماً صريحاً	نعم، يملك اسماً صريحاً	نعم، يملك اسماً صريحاً داخل نطاقه	لا يملك اسماً صريحاً في الشيفرة
الحاجة لكائن خارجي لإنشائه	نعم، يتطلب عادةً كائناً من الكلاس الخارجي Outer Class	لا، يمكن إنشاؤه عبر اسم الكلاس الخارجي مباشرة	لا يمكن إنشاؤه إلا داخل النطاق الذي عُرّف فيه، وقد يرتبط بكائن خارجي إذا كان داخل دالة غير ساكنة	يُنشأ مباشرة في موضعه، اعتماداً على كلاس أب أو واجهة
صلاحية الوصول للـ Outer Class	يمكنه الوصول إلى عناصر الكلاس الخارجي، بما فيها private	يمكنه الوصول مباشرة إلى العناصر الساكنة، أما عناصر الكائن غير الساكنة فتحتاج إلى مرجع لكائن من الكلاس الخارجي	يمكنه الوصول إلى عناصر النطاق المتاحة له، وإلى عناصر الكلاس الخارجي إذا كان ضمن سياق يسمح بذلك	يمكنه الوصول إلى عناصر النطاق المتاحة له، وإلى عناصر الكلاس الخارجي إذا كان ضمن سياق يسمح بذلك
إمكانية إعادة الاستخدام	نعم، متوفر على مستوى الكلاس	نعم، متوفر على مستوى الكلاس	نعم، ولكن داخل النطاق الذي عُرّف فيه فقط	لا يُعاد استخدامه كنوع مستقل، لأنه لا يملك اسماً صريحاً، ويُستخدم غالباً في موضع تعريفه



انتهى مقرر البرمجة 2 .. لا تنسونا من صالح دعائكم